

USO DE INTELIGENCIA ARTIFICIAL

DECLARACION EVALUACION N°2

NOMBRE: Osvaldo Uhart Molina

CARRERA: Analista Programador

ASIGNATURA: Programación Backend

PROFESOR: Brayan Mansilla Pérez

FECHA: 08 de octubre del 2026



Indice

1. Introducción.....	3
2. Uso de Inteligencia Artificial.....	3
2.1 Parte 1. Registro de consultas.....	4
2.2 Parte 2. Explicación personal del proceso.....	6
3. Conclusión.....	6

Introducción

En este documento presento cómo utilicé la inteligencia artificial como apoyo durante la Evaluación Sumativa 2 de Programación Backend. El trabajo consistió en continuar el catálogo online de la ferretería El Tornillo, desarrollado en Django para la Evaluación 1, e incorporar una base de datos SQLite para almacenar sus productos. Esta etapa incluyó la definición del modelo Producto, la aplicación de migraciones y la configuración de Django Admin para crear, editar y eliminar registros.

La IA se utilizó para organizar el desarrollo según la pauta, proponer el modelo, resolver dificultades de configuración y generar una fixture con 40 productos de ferretería. También apoyó la adaptación del catálogo para consultar la base de datos mediante el ORM. A continuación, se presentan las consultas realizadas, las respuestas obtenidas y los ajustes aplicados antes de integrarlas al proyecto.

Uso de Inteligencia Artificial

Proyecto: Catálogo Online, variante A (Ferretería, 40 productos).

Repositorio: <https://github.com/OsvaDev-HUB/Evaluacion-2-Programacion-Backend-.git>

Alcance: Reutilización del proyecto de la prueba 1, configuración de SQLite, modelo Producto, migraciones, administración en /admin, fixture de 40 productos generada con IA y listado desde la base de datos mediante el ORM.

Inteligencia artificial elegida: MiMo-V2.6-Flash (modelo gratuito).

Organización del trabajo: Desarrollo por las etapas de la pauta, entorno, modelo y base de datos, Django Admin, poblamiento y listado, y entrega. El registro incluye el prompt textual, la respuesta obtenida y los ajustes realizados antes de integrarla.

Parte 1: Registro de las consultas

Consulta 1: Configuración del entorno

Prompt: Realiza un análisis de la pauta de evaluación (PDF) para planificar el trabajo por etapas y revisar el estado inicial del entorno de desarrollo en el que vamos a trabajar.

Respuesta: resumen de las etapas de la pauta. Indicó que el entorno virtual no tenía pip porque faltaba el paquete `python3.14-venv` del sistema.

Qué usé o modifiqué: instalé el paquete, volví a crear el venv e instalé Django. Copié el código de mi prueba 1 sin venv, .git ni archivos temporales, y ordené la documentación en `Docs/Evaluacion 1` y `Docs/Evaluacion 2`.

Consulta 2: Modelo de datos

Prompt: Necesito que hagas una definición del modelo de datos de la variante A (nombre, categoría, precio, stock) con tipos de campo adecuados y método `__str__`, además de los pasos para generar y aplicar migraciones.

Respuesta: propuesta del modelo Producto con nombre y categoria (CharField), precio y stock (PositiveIntegerField), descripcion opcional y método `__str__`.

Qué usé o modifiqué: usé el modelo propuesto. Mantuve descripcion porque la vista de detalle de la prueba 1 ya la mostraba. El primer makemigrations respondió "No changes detected" porque no había guardado el archivo.

Consulta 3: Permisos en Django Admin

Prompt: Realiza un diagnóstico de por qué el superusuario no ve ningún modelo en /admin después de registrar Producto en admin.py.

Respuesta: el backend de autenticación que venía de la prueba 1 (`UsuariosJSONBackend`) era el único configurado y dejaba a todos los usuarios con `is_superuser=False`.

Qué usé o modifiqué: agregué `django.contrib.auth.backends.ModelBackend` antes del backend de la prueba 1 en `AUTHENTICATION_BACKENDS` y creé un superusuario con un nombre distinto de admin.

Consulta 4: Poblamiento inicial con IA (fixture)

Prompt: Necesito que hagas una generación de una fixture JSON de Django con 40 productos de ferretería chilenos realistas para el modelo `catalogo.producto` (campos nombre, categoria, precio entre 1000 y 150000, stock entre 0 y 50, descripcion), manteniendo los nombres y el orden de la prueba 1, y su carga con `loaddata`.

Salida obtenida: archivo `catalogo/fixtures/productos.json` con 40 objetos en formato model, pk y fields, con los campos nombre, categoria, precio, stock y descripcion.

Ajustes para cargarla en mi BD:

- El modelo se indica como `catalogo.producto`, en el formato `app.modelo` y en minúscula.
- Se mantuvieron los mismos nombres y el mismo orden de productos de la prueba 1, porque las fotos del sitio están asociadas al id (`producto-01.webp` a `producto-40.webp`).
- Precios y descripciones nuevos. El stock quedó entre 0 y 50, con 8 productos agotados.
- El primer `loaddata` falló con `no such table: catalogo_producto`, porque la migración estaba creada pero no aplicada. Se ejecutó `migrate` y luego `python manage.py loaddata productos`, que cargó 40 objetos.

Consulta 5: Listado desde la base de datos

Prompt: Reemplaza la lectura de `data/catalogo.json` en la vista por una consulta a la base de datos con el ORM, sin romper los templates existentes.

Respuesta: nueva versión de `cargar_productos()` en `catalogo/views.py` que consulta el modelo `Producto`.

Qué usé o modifiqué: `cargar_productos()` ahora retorna `list(Producto.objects.order_by("id").values())`. Se usa `.values()` porque devuelve diccionarios con las mismas claves que tenía el JSON, así el resto de las vistas y templates de la prueba 1 sigue funcionando sin cambios. Se quitaron los imports que quedaron sin uso.

Parte 2: Explicación personal del proceso

En esta evaluación usé la IA como apoyo para ir avanzando por etapas según la pauta. Al principio le pedí que leyera el PDF y me ordenara los pasos, y ahí me di cuenta de que mi entorno virtual estaba mal creado, porque no tenía pip ni el archivo activate. El problema era que a mi Python 3.14 le faltaba el paquete python3.14-venv, así que lo instalé y volví a crear el venv.

Reutilicé el código de mi prueba 1. El modelo Producto me lo propuso la IA y lo usé casi igual, pero le dejé el campo descripción porque mi página de detalle ya lo mostraba. Tuve dos errores que me enseñaron cosas, primero makemigrations no detectaba cambios porque no había guardado el archivo y después el loaddata falló porque nunca ejecuté migrate. Ahí entendí que makemigrations solo crea el archivo y migrate es el que crea la tabla en la base de datos.

El problema que más me costó fue el admin, porque entraba pero decía que no tenía permisos. Era el login que hice en la prueba 1, que leía los usuarios desde un JSON y dejaba a todos sin superusuario. Lo solucioné agregando el ModelBackend de Django. La fixture la generó la IA, pero le pedí mantener los mismos nombres de la prueba 1 para que las fotos de cada producto siguieran calzando con su id. También aprendí que con .values() el ORM devuelve diccionarios, y por eso mis templates siguieron funcionando sin cambiarlos.

Conclusión

El desarrollo de esta evaluación permitió incorporar la persistencia de los productos en SQLite y su administración desde Django Admin. El catálogo conserva la interfaz de la Evaluación 1, pero obtiene sus datos mediante el ORM. La fixture generada con IA permitió cargar los 40 productos requeridos, manteniendo la relación entre sus identificadores y las imágenes existentes.

El proceso también permitió comprender la diferencia entre generar y aplicar migraciones, la necesidad de contar con las tablas antes de cargar los datos y el papel del backend de autenticación en los permisos del superusuario. La IA aportó propuestas y orientación para resolver esas dificultades; su integración requirió ajustar los nombres, el formato de la fixture y la lectura de los productos. Esta experiencia refuerza la importancia de revisar y comprender las respuestas de la IA antes de utilizarlas en una aplicación.



APRENDER HACIENDO A LO LARGO DE LA VIDA

ACREDITADO

**7 AÑOS
MÁXIMA
EXCELENCIA**



INSTITUTO PROFESIONAL INACAP

ACREDITADO • NIVEL DE EXCELENCIA

- Gestión Institucional • Docencia de Pregrado
- Vinculación con el Medio
- Próxima Acreditación Septiembre 2030

www.inacap.cl